Laboratório Prolog · Aula 13

Pedro Massaki Carnelossi · RA 24036481-2 · Turma ESOFT6S-B
Gerado em 23/10/2026, 16:25:17 · tempo registrado no cronômetro: 13:36

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) responde L = scheme. paradigma(java, logico) responde false

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem uma variável. Ela pergunta “qual é o paradigma de prolog?”, e o Prolog procura o valor de P que torna a consulta verdadeira (P = logico). paradigma(prolog, logico) não tem variável. Ela pergunta “prolog é lógico?” e só pode responder verdadeiro ou falso.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que “o programa não consegue provar isso”. O false significa que nenhum fato ou regra da base prova a consulta: é a hipótese do mundo fechado. Neste caso coincide com a realidade, mas se a base não tivesse nenhum fato sobre Java, paradigma(java, orientado_a_objetos) também daria false, mesmo sendo verdade.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas. (mae(M, F), mulher(F)) terá 4 pares: lucia–tereza, marta–joana, tereza–bia e joana–ana.

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
```

```
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mae(A,B),homem(B).
   mae(lucia,vern),homem(vern)
   mae(marta,paulo),homem(paulo)
   mae(joana,jake),homem(jake)
   mae(dora,leo),homem(leo)
?- homem(A),mulher(A).
   false.
?- mae(joana,jake),mae(joana,ana).
   mae(joana,jake),mae(joana,ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta significa “quais mães têm filhas?”: pares em que Mae é mãe de Filha e Filha é mulher. Teve 4 respostas: lucia–tereza, marta–joana, tereza–bia e joana–ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

Isso se explica pela hipótese do mundo fechado: o que não está na base e não pode ser provado é tratado como falso. Em situações reais, “não consta” vira “não é verdade”. Por exemplo, um sistema pode negar um benefício ou um acesso só porque um cadastro está incompleto, e não porque a pessoa não tem direito.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas saíram na ordem lucia, marta, tereza, joana, bia, ana, dora. É a ordem em que os fatos estão escritos no arquivo, porque o Prolog percorre a base de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) responde X = paulo, que é irmão de joana. primo(leo, X) responde jake e ana, porque paulo (pai de leo) é irmão de joana (mãe deles).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

f(X, X) = f(a, b), [A|B] = [] e 2 + 3 = 5. Só f(X, X) = f(a, Y) unifica, com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
```

```
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
?- irmao(A,B).
   irmao(vern,tereza)
   irmao(tereza,vern)
   irmao(joana,paulo)
   irmao(paulo,joana)
   irmao(jake,ana)
   irmao(ana,jake)
?- tio(A,B).
   tio(vern,bia)
   tio(paulo,jake)
   tio(paulo,ana)
?- ancestral(A,leo).
   ancestral(dora,leo)
   ancestral(paulo,leo)
   ancestral(lucia,leo)
   ancestral(marta,leo)
   ancestral(osvaldo,leo)
   ancestral(vern,leo)
?- primo(A,B).
   primo(bia,joana)
   primo(bia,paulo)
   primo(jake,leo)
   primo(ana,leo)
   primo(joana,bia)
   primo(paulo,bia)
   primo(leo,jake)
   primo(leo,ana)
[vern,tereza,joana,paulo,jake,ana,leo,bia]
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

ancestral precisa de duas cláusulas porque uma para a recursão e a outra a faz avançar.

Caso base: ancestral(X, Y) :- progenitor(X, Y). O progenitor direto é ancestral, e é aqui que a recursão para.
Caso recursivo: ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y). X é ancestral de Y se for progenitor de alguém (Z) que é ancestral de Y.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

X \== Y, irmao(jake, A) passou a responder também irmao(jake, jake), e primo(leo, A) ganhou primo(leo, leo). Com X = Y, “mesma mãe e mesmo pai” é sempre verdadeiro, então cada pessoa vira irmã de si mesma. A meta X \== Y exclui esse caso.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X, X) = f(a, b), X teria que valer a e b ao mesmo tempo, o que é impossível. Em f(X, X) = f(a, Y), o primeiro argumento liga X = a, e o segundo exige X = Y, então Y = a. Resultado: X = a, Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não. Em Java ou Scheme, uma função tem entradas e saída fixas: avos(neto) devolve os avós, e para achar os netos seria preciso escrever outra função, netos(avo). Em Prolog, avo é uma relação: qualquer argumento pode ser variável, e a unificação com backtracking encontra os valores que faltam, nas duas direções.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

4 respostas: [] + [1,2,3], [1] + [2,3], [1,2] + [3] e [1,2,3] + [].

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
```

```
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
?- concatena(A,[b,c],[a,b,c]).
   concatena([a],[b,c],[a,b,c])
?- inverte([a,b,c,d],A).
   inverte([a,b,c,d],[d,c,b,a])
?- tamanho([x,y],A).
   tamanho([x,y],2)
?- membro(A,[]).
   false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Em tamanho:

Caso base: tamanho([], 0).
Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Uma lista de 3 elementos pode ser cortada em 4 pontos (antes do 1, entre 1 e 2, entre 2 e 3, depois do 3), por isso há 4 respostas. O backtracking tenta a 1ª cláusula (X = []) e depois a 2ª (a cabeça vai para X), repetindo na cauda. O anexar do Scheme só calcula list1 + list2 → resultado. Ele não consegue partir do resultado para descobrir as partes, nem gerar todas as divisões.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking, junto com a unificação. A 1ª cláusula unifica X com a cabeça (a). Quando se pede outra resposta, o Prolog volta e tenta a 2ª cláusula, que repete a busca na cauda (b, depois c).

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 deixa X valendo a estrutura 2+3, sem calcular. X is 2 + 3 dá 5. fatorial(N, 120) dá erro de instanciação.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
?- A is 5*4+2.
   22 is 5*4+2
?- A=5*4+2.
```

```
   5*4+2=5*4+2
?- fatorial(10,A).
   fatorial(10,3628800)
?- fib(20,A).
   fib(20,6765)
?- fatorial(A,120).
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica sem calcular: X = 2 + 3 faz X valer a estrutura 2+3, e 2 + 3 = 5 dá false. is calcula o lado direito e unifica com o esquerdo: X is 2 + 3 dá X = 5. =:= compara os valores numéricos dos dois lados, que precisam estar conhecidos: 5 =:= 2 + 3 dá true e 4 =:= 2 + 3 dá false.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

concatena só usa unificação, que funciona em qualquer direção. fatorial usa N > 0 e N1 is N - 1, que exigem N já conhecido. Com N livre o Prolog não tem como calcular e lança instantiation_error. A aritmética é avaliação, como uma função de mão única, e não uma relação.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: a estrutura (caso base 0 resulta em 1, caso recursivo n vezes o fatorial de n - 1) e a repetição por recursão, sem laço. Diferente: Haskell devolve o valor, enquanto no Prolog o resultado é um argumento extra (F). O Prolog precisa da guarda N > 0 explícita, porque o backtracking tentaria a 2ª cláusula também com N = 0, e precisa do is para calcular e de uma variável nova (N1), já que as variáveis não mudam de valor.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dá false. maximo2(5, 3, M) dá duas respostas (M = 5 e M = 3), porque a 2ª cláusula não tem teste. maximo3(5, 3, 3) dá true, o que é errado.

Saída de controle.pl e mensagem de ancestral_ruim

```
=== controle.pl ===
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
```

```
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)
?- maximo3(8,3,A).
   maximo3(8,3,8)
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- maximo2(8,3,A).
   maximo2(8,3,8)
   maximo2(8,3,3)

=== laco_infinito.pl ===
ancestral_bom(osvaldo, jake)?
```

```
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações. Com 2 cores, nenhuma: PR, SP e MS fazem fronteira entre si (formam um triângulo) e precisariam de 3 cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog faz amigo(X), que dá ana, e amigo(Y), que também dá ana. Em "tenta ana e ana" a meta que falhou foi X \== Y, e ele voltou ao último ponto de escolha, amigo(Y), tentando Y = bia. Falhou em gosta (ana gosta de cafe e bia de cha), então tentou Y = caio, que também gosta de cafe: 1ª resposta. Quando as opções de Y acabaram, voltou a amigo(X) (bia, depois caio) e encontrou caio e ana: 2ª resposta.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro2, o \+ casado(X) roda com X ainda livre. Como casado(X) é provável (X = vern), o \+ falha e a regra inteira falha: ela testa "não existe ninguém casado", e não "X não é casado". solteiro1 primeiro liga X com homem(X) e só depois nega. Osvaldo aparece como solteiro porque não há o fato casado(osvaldo): pelo mundo fechado, o que não se prova é falso. O programa não sabe nada sobre ele ser pai de vern.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

A cabeça da 1ª cláusula, maximo3(X, Y, X), exige que o 3º argumento seja igual ao 1º (5). Como é 3, a unificação da cabeça falha antes de chegar ao corte, e o corte nunca executa. Então o Prolog tenta a 2ª cláusula, maximo3(_, Y, Y), que aceita o 3 sem nenhum teste. Correção (unificação depois do corte), testada: maximo3(5, 3, 3) passa a dar false.

```
maximo3(X, Y, Z) :- X >= Y, !, Z = X.
maximo3(_, Y, Y).
```

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim chama a si mesmo primeiro, com o mesmo X e sem consumir nenhum fato (recursão à esquerda). A busca em profundidade desce infinitamente sem nunca chegar a progenitor, e a pilha estoura. ancestral_bom consome um progenitor antes de recursar, então desce uma geração por chamada e termina. Logicamente as duas são iguais (o "e" é comutativo), mas a ordem em que o Prolog executa as metas muda o comportamento.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Encontrou 24 colorações com 3 cores, 0 com 2 cores (deu false) e 1728 com 4 cores. É declarativo porque só descreve o que é uma coloração válida (cada estado recebe uma cor da paleta e vizinhos têm cores diferentes). Não diz como procurar, não tem laço nem algoritmo: quem faz a busca é o próprio Prolog, por backtracking (gerar e testar).

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar é uma função: recebe list1 e list2 e devolve a junção, numa única direção. No lógico, concatena(A, B, C) é uma relação entre três listas: o mesmo código junta, separa (concatena(X, Y, [1,2,3]) dá as 4 divisões) e testa, porque o Prolog resolve por unificação e backtracking.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Controle da ordem de resolução: em laco_infinito.pl, ancestral_ruim estoura a pilha mesmo sendo logicamente igual a ancestral_bom; em controle.pl, o corte de maximo3 deu uma resposta errada (maximo3(5, 3, 3) = true).
2) Negação como falha e mundo fechado: em controle.pl, solteiro2 dá false só por causa da ordem das metas, e solteiro1 diz que osvaldo é solteiro só porque falta o fato casado(osvaldo).

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras e busca: sistemas especialistas, regras de elegibilidade e de negócio, consultas dedutivas sobre relações (genealogia, grafos), quebra-cabeças e problemas de restrição (como o mapa e o SEND + MORE) e análise de linguagem.

Nesses casos basta descrever o problema e o motor faz a busca. Não escolheria para sistemas com muita entrada e saída, interface gráfica, estado mutável, cálculo numérico pesado ou exigência de desempenho previsível. Nesses casos o Java tem ecossistema, tipagem, desempenho e ferramentas que o Prolog não tem.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
```

```
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

E1 Regras de família: código escrito

```
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|Cauda], X) :- ultimo(Cauda, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.
pares([], []).
pares([H|T], [H|P]) :- H mod 2 =:= 0, pares(T, P).
pares([H|T], P) :- H mod 2 =\= 0, pares(T, P).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
```

```
caminho(A, B, Visitados) :- liga(A, C), \+ membro(C, Visitados), caminho(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), Lista), soma(Lista, Total).
```